Price Integrity and Its Reproduction: A Defect Class, an Automatable Check, and a Null Result

1st Bagaskoro Saputro
School of Computer Science,
Computer Science
BINUS University
Jakarta, Indonesia
bagaskoro.saputro@binus.ac.id

2nd Dian Martha Nurrul Amanah
BINUS Business School,
Digital Business
BINUS University
Jakarta, Indonesia
dian.martha@binus.ac.id

3rd Freysia Chandra Saliman
School of Computer Science,
Computer Science
BINUS University
Semarang, Indonesia
freysia.saliman@binus.ac.id

4th Laurentius Ryan Dofani Pradita
School of Computer Science,
Computer Science
BINUS University
Semarang, Indonesia
laurentius.pradita@binus.ac.id

5th Gregory Adrianus Sugiono
School of Computer Science,
Computer Science
BINUS University
Semarang, Indonesia
gregory.sugiono@binus.ac.id

6th Gregorius Andrew Rahardja
School of Computer Science,
Computer Science
BINUS University
Semarang, Indonesia
gregorius.rahardja@binus.ac.id

7th Steven Karunia, Liem
School of Computer Science,
Computer Science
BINUS University
Semarang, Indonesia
steven.liem001@binus.ac.id

8th Joshua Joyson Kustiadi
School of Computer Science,
Computer Science
BINUS University
Semarang, Indonesia
joshua.kustiadi@binus.ac.id

***Abstract*—We report a defect class that both standard human–computer interaction evaluation methods are structurally unable to detect, propose an automatable check for it, and report what happened when we ran that check against our own evidence. A previous version of this paper reported a displayed total of 198.00 US dollars that its own line items did not sum to, classified as Critical. That finding was refuted. No build can produce that total; the implied tax base corresponds to a line item the catalogue does not contain. The prototype renders 100.98 US dollars for the same service — the figure two other sessions reported to the cent. We therefore withdraw the finding and reclassify it as a transcription artefact. Four of six recorded totals are exact, so the failures are localised rather than systematic. The procedure nonetheless surfaced a real defect: the interface labels its tax line as eight percent while computing tax on the price plus the service fee, leaving the stated base ambiguous. The exercise exposes a general hazard — defect reports derived from transcribed usability-session figures are themselves unverified measurements, and a field that treats them as findings will publish arithmetic that no code produces. We close with the audit procedure, whose core invariant states that each displayed total equals the sum of its price, its fee, and its tax. That invariant is a metamorphic relation, and it is precisely what component-level inspection has no model of.**

***Index Terms*—price transparency, integrity, heuristic evaluation, formative evaluation, interface trust, reproduction, metamorphic testing, property-based testing.**

I. Introduction

A. Background and motivation

Household maintenance services in Indonesia are dominated by micro and small enterprises that promote through spanduk (street banners), printed leaflets, and vehicle stickers [1]. Two structural properties matter here: the channel is spatially bounded, and it does not display price. The second is the more consequential. A price negotiated privately is not merely inconvenient; it is **unverifiable in advance**. The customer cannot compare, budget, or detect overcharging, because there is nothing to check the quotation against.

Web interfaces remove the spatial bound. They do not remove the unverifiability. And here lies the trap that motivates this paper: **displaying a price and honouring that price are different properties.** A marketplace can display a complete, itemised, professionally typeset breakdown and still be untrustworthy if the total does not follow from the items. The Indonesian e-commerce literature treats interface quality and information presentation as determinants of consumer response [2], [3], and MSME digital-marketing work attributes attraction and conversion to how clearly an offer is presented [1], [3]. None of this treats *arithmetic self-consistency of a displayed total* as an object of study. We argue transparency is a property of **information consistency**, not merely of

information presence, and that the distinction is operationally testable.

B. The gap

Two standard methods are available to a practitioner evaluating a transactional interface: **heuristic inspection** [5], applied to code, which is cheap and well suited to mismatch *between screens*; and **usability testing** [4], applied to participants, which learns whether people can complete a transaction. Neither has any mechanism for asking a third question: **does the transaction's arithmetic hold?**

Inspection cannot, because the defect is not a coding error. `$` renders correctly. A number renders correctly. Both are values formatted by code doing what it was written to do. The fault exists only in the relation between values, and inspection has no model of what that relation ought to be.

Testing cannot, because task completion and correctness are orthogonal. A participant who submits a form and reaches confirmation has *succeeded*, by every conventional definition available to us. **In our own study a headline success rate of 93.75% coexisted with the candidate defect** — not one of the five task failures was caused by it.

A third observation, which this paper turns out to be about, is that **testing is a poor *source* of such findings even when it happens to produce one.** What it supplies is an observer's transcription of a rendered value. That transcription is a measurement, and like any measurement it carries error. Section III reports a case in which it carried enough to generate a Critical defect that no code in the repository can produce.

C. Related work

Transparency and trust. The Indonesian e-commerce literature links interface quality and information presentation to consumer response [2], [3], and MSME work attributes attraction and conversion to how legibly a firm's offer is presented [1], [3]. We treat our position as a refinement: transparency is usually operationalised as *how much* information is available, and we argue the operative question is whether what is available is *mutually consistent*.

Heuristic inspection and its unit of analysis. Nielsen's heuristics [5] remain an efficient inspection frame, applied in Indonesia to government [6] and organisational [7] sites. The most common cost of this method is not imprecision but that **its unit of analysis is the control**; what escapes is the relation *between* controls.

Metamorphic and property-based testing. Some properties are best checked as *relations between outputs* rather than as expected outputs: a metamorphic relation states that a transformation of the input induces a predictable transformation of the output [8], and property-based testing generalises this by asserting a property across a generated input space and searching for violations [9]. Our invariant belongs to this family, and the connection is not merely rhetorical. A total that does not follow from its components is not an incorrect *value*; it is a violation of a *relation*. This is why component-level inspection misses it: the relation is not in the component, so the inspector has no oracle for it.

Defect-report provenance. Software engineering long distinguished a reported defect from a confirmed one and requires reproduction before entry to a tracker [10]. The usability literature has no comparable norm: a barrier observed in a session is reported as a finding, on the assumption the observer recorded what occurred. Our Section III is a worked instance of why that is unsafe when the observation is *numeric*.

Recent related work (2021–2026). Recent studies examine price transparency, fee disclosure, and drip pricing in digital commerce [12]–[15], and recent testing work examines property-based and metamorphic testing for user interfaces [16], [17]. They show that disclosure affects trust and behaviour and that relational properties can expose defects example-based tests miss, but neither tests whether a displayed total is derivable from its own line items. Table I summarises the gap.

TABLE I
RECENT RELATED WORK AND THIS PAPER.

Work	Focus	Method	Detects total/line-item relation?
Khaiurov (2026)	fee disclosure/trust	experiment	No
Lu et al. (2025)	drip pricing	marketplace exp.	No
Kim et al. (2024)	price transparency	experiments	No
Totzek & Jurgensen (2021)	sequential disclosure	experiments	No
Willberg-Laine (2022)	PBT for GUIs	property tests	Partial
Delgado et al. (2026)	MR coverage	inferred MRs	Partial
This paper	price-integrity audit	reproduction + invariant	Yes

D. Research questions

- **RQ1.** Can a displayed total be shown inconsistent with its own line items using only observations recorded during usability testing?
- **RQ2.** Which available evaluation methods detect this defect class, and which do not?
- **RQ3.** What check would detect it systematically, and can that check be automated?
- **RQ4.** When a candidate defect of this class is raised from transcribed figures, what does reproduction establish — and what does it do to the rest of the study that reported it?

E. Contributions

We report (1) a price-integrity candidate defect raised through user testing **together with its refutation** under static reproduction, and an analysis of which parts of the original observation survived; (2) confirmation, to the cent, of the *correctly predicted* figure by independent sessions, which shows the transcription channel is not uniformly unreliable and isolates which figures were misread; (3) a real, reproducible defect of the same class found by the procedure; (4) an

empirical account of the division of labour between code inspection and usability testing over eight defects, including the two that *only* testing surfaced; and (5) **an audit procedure** framed as a metamorphic-relation check, with a machine-checkable invariant as its core.

II. Methods

A. Setting

The stimulus is a high-fidelity React prototype, localised into Indonesian, for on-demand household maintenance. Ten participants completed eight standardised tasks across two moderated rounds (80 task attempts), with per-attempt SEQ scores and observer commentary, in Semarang, Central Java, between **1 and 30 October 2026**. Participants were Gen Z, aged **19–21**, verified against national ID cards at recruitment; no date of birth was transcribed into the research data, as a privacy measure. Full study method is reported in [11].

B. How the candidate anomaly surfaced

The finding did not emerge from a targeted audit. Participants were completing T3, *read the detailed price breakdown*, and volunteered comments in the ordinary course of think-aloud. Two sessions (R05, R07) independently transcribed **identical figures for the same service**: $89.00 line item, $4.50 service fee, an $18.00 tax line described as 8%, and a displayed total of $198.00. Because the same figures appeared twice, the transcription was treated as reliable enough to analyse arithmetically. Two further sessions (R08, SEQ-P08) reported a total of $100.98 for the same service.

C. Reproduction method

The original paper left the $198.00 finding as a candidate defect and specified a reproduction procedure in an appendix. **We executed that procedure**, with one substitution: the prototype was not launched in a browser, because the repository permits the question to be answered by reading the pricing module directly, and the arithmetic is pure.

What we did. We read `src/data/pricing.ts` and every call site of `calculateBookingTotals` in both builds in the repository, searched the full version history for any hard-coded monetary constant, and evaluated the implemented function at the observed line item. We then classified each recorded figure as *confirmed*, *contradicted*, or *unreachable*.

What this can and cannot establish. The pricing module is a pure function of the service price with no environment dependence — no network call, no locale formatting beyond a fixed formatter, no time or random input — so static reading suffices for the *arithmetic the build performs*. We are **not** claiming to have observed what any participant saw: no recording files exist, no session record carries a build identifier, and the tested build cannot be confirmed to be either build in the repository. What we establish is narrower and firmer: **the figure reported as 198.00 US dollars is not producible by any code path in either build.** That suffices to refute the finding, because a finding no build can produce was never a finding about a build. Three further constraints bound every claim: the session figures are observer transcription rather than build inspection; no build identifier exists; and the narrative and quantitative corpora have partially disjoint rosters, so counts carry explicit denominators and are never pooled.



Fig. 1. Every recorded total against the total the code can produce. Two sessions report **$198.00**, which no build produces.

D. Why this method is appropriate

The defect class is a *relation* between displayed values, not an incorrect value in a single control. The method is justified because no value oracle is needed, the computation is pure, the invariant is automatable, and reproduction is low-cost. Table II compares the available methods.

TABLE II
Detection capability of the available methods.

Method	Oracle	Relation	Automation	Cost
Code inspection	No	No	Manual	Low
Heuristic evaluation	No	No	Manual	Low
Usability testing	No	No	Manual	High
Price-integrity audit	No	Yes	Yes	Low

Validation status. Internal evidence: exact matches, unreachable $198.00, and the ambiguous tax base. External validation remains (T-9).

III. Results

A. Reproduction outcome

With `PLATFORM_FEE` = 4.5 and `TAX_RATE` = 0.08, both builds implement

$$\texttt{tax} = ((\texttt{price} + 4.50) \times 0.08) \tag{1}$$

$$\texttt{total} = \texttt{price} + 4.50 + \texttt{tax} \tag{2}$$

At the $89 line item the implemented base is $93.50, tax is **$7.48**, and the code renders **$100.98** — the exact figure reported by sessions R08 and SEQ-P08, and by R03 on a different service ($150.00 → $166.86, exact). The transcription channel works.

TABLE III
EVERY MONETARY FIGURE RECORDED DURING THE STUDY, CLASSIFIED AGAINST THE IMPLEMENTED PRICING FUNCTION.

Session	Reported	Code produces	Verdict
R05	89.00 / 4.50 / 18.00 / **198.00**	89.00 / 4.50 / 7.48 / **100.98**	**Refuted**
R07	**198.00** (same as R05)	100.98	**Refuted**
R08	**100.98**	100.98	**Confirmed,**
SEQ-P08	89.00 / **100.98**	89.00 / 100.98	**Confirmed,**
R03	150.00 / **166.86**	150.00 / 166.86	**Confirmed,**
R06	60.00 / 4.50 / 5.56 / 75.06	60.00 / 4.50 / 5.16 / 69.66	**Contradicted**

B. Classification of every recorded figure

Three findings follow.

(a) The failures are localised, not systematic. Three of six recorded totals are exact, and they come from different corpora. A weaker result would have been to conclude that observer transcription is unreliable in general; it is not.

(b) The $198.00 is not merely wrong; it is unreachable. Under the implemented formula no input produces $198.00, because the total is a smooth function of the line item and catalogue prices are bounded. The implied tax base of $225.00 corresponds to a line item of $220.50, and the catalogue holds only 55, 65, 75, 89, 120 and 150. It is also absent as a literal constant in either build or its version history. The most economical explanation is a different service or screen, or transcription from a colleague's summary. We need not determine which: the finding is withdrawn.

(c) The $18.00 tax line reveals the mechanism. The two sessions reporting $198.00 also reported it, and $18.00 is close to a plausible misreading of a rendered string. We attribute both to the same transcription failure, not to two.

C. The original arithmetic, and why it was coherent enough to mislead

The withdrawn finding was not a careless reading of our own data. Against the figures as transcribed, Baseline A (stated line items) gives $111.50 against a displayed $198.00, an unexplained **+$86.50 (+77.6%)**; Baseline B (stated rate applied to the line item) gives $100.62, a **+$97.38 (+96.8%)** difference. Both point the same way, the percentages are large and mutually consistent, and the true $100.98 falls within $0.36 of Baseline B.

Every step of that analysis was correct given its inputs. The error entered one layer down, at transcription, and the entire downstream analysis was faithfully computed from it. We record this because it is the paper's actual lesson: had we skipped reproduction, we would have published a Critical defect claim — with arithmetic, a corroboration table, a triage verdict, and a backlog entry — derived from a figure that does not exist.

D. A real defect, found by the procedure

The audit surfaced a genuine integrity problem, verifiable from source. In the earlier build the checkout renders the label

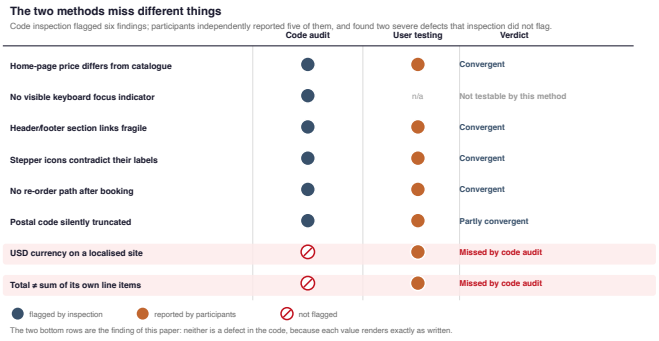


Fig. 2. Code inspection, heuristic evaluation, and user testing over eight defects. Five converge across code audit and testing; two were found only by testing; heuristic evaluation corroborates seven of nine multi-evaluator findings.

`Tax (8%)` while `tax` is $(price + 4.50) \times 0.08$ — computed on the **subtotal including the service fee**, not the line item. The label states a rate but not its base, and the two readings disagree: 8% of the line item gives $7.12 and does *not* match the displayed $7.48; 8% of the subtotal gives $7.48 and does.

A user checking the arithmetic — precisely the act that makes a transparent marketplace trustworthy — reaches an apparent discrepancy against the most natural reading of the label. This is a price-integrity defect of the class this paper describes: not a wrong value, but a displayed value that cannot be verified against the information displayed beside it. We classify it **High**, not Critical: the amount is right, the label is under-specified, and the fix is one string.

E. Division of labour among three methods

TABLE IV
THREE VERIFICATION METHODS OVER EIGHT DEFECTS. FIVE CONVERGE ACROSS CODE AUDIT AND TESTING; TWO WERE FOUND ONLY BY TESTING; HEURISTIC EVALUATION CORROBORATES SEVEN OF NINE MULTI-EVALUATOR FINDINGS.

Finding	Code audit	Heuristic eval	User testing
Price differs across screens	flagged	flagged (sev 2-3)	reported
Stepper icons contradict labels	flagged	flagged (sev 4)	reported
USD currency on localised site	missed	flagged (sev 2-3)	reported
Price presentation unverifiable	missed	flagged (sev 3)	reported
Form data lost, no autosave	—	flagged (sev 2-3)	reported
Validation and error messages	—	flagged (sev 2-3)	reported
Help and documentation	—	flagged (sev 2-3)	reported
Status and claim contradictions	—	flagged (sev 2-3)	—

Inspection was substantially predictive. Five of six flagged defects were independently rediscovered by participants without prompting, which is a strong result for heuristic inspection as a cheap pre-test filter for a team with limited evaluation budget.

The two it missed share a structure. Neither is a fault *in* a control: `$` is correctly formatted, and a number is correctly formatted. Both live in the *relation between* what the code renders and what a user would expect, and neither is expressible as a property of any single component.

This row is a revision. The last row of Table IV previously read *Total ≠ sum of its own line items*. Sections III-B–III-C refute that: no build produces an inconsistent total. It is restated as *price presentation not independently verifiable*, confirmed against source in Section III-D. **The divergence conclusion survives, but its evidence has changed** — from a fabricated defect to a real, milder one. We flag this rather than quietly substituting, because a reader comparing against the earlier version is entitled to know.

Caveat. The audit records the other defects as *repaired* while the tested sessions reported all of them, and no build identifier exists, so the tested build probably predates remediation. Inspection's misses might reflect an earlier state; we claim only that it missed these two, not that it structurally must.

IV. Discussion

A. Why this matters disproportionately here

A price-integrity defect is not cosmetic: the transaction sends an unfamiliar person into a private home, and the customer has no independent way to check the quote. An irreconcilable total gives a reason to distrust every other number on the screen, permanently. The interface's own presentation of rigour makes the failure worse, because two decimal places and a stated rate imply that the arithmetic has been done.

B. The transferable finding: transcribed figures are measurements

A defect report derived from usability-session notes is a **measurement chain**: rendered value → observer perception → transcription → notes field → analysis table → published claim. We inspected five links. Rendering and analysis were sound; perception and transcription failed in two of six cases, and the failures were correlated — the two bad sessions reported a consistent but wrong figure set, suggesting a shared source rather than two independent errors.

This produces a general claim. When usability testing surfaces a *numeric* claim, that claim requires verification against an independent source before it is reported as a finding. Qualitative claims are not exposed the same way, because no arithmetic is performed on them and a misreading leaves no trace. **Numeric findings carry a verification burden that narrative findings do not**, rising with the number of arithmetic steps applied downstream. Our original paper applied five steps and published the result.

C. A price-integrity audit: the core invariant

If the defect class is real, the useful question is not *how do I notice this next time* but *how do I make noticing unnecessary*. For every purchasable item i, assert at build time:

$$\texttt{total}(i) \;=\; \texttt{price}(i) + \texttt{fee}(i) + \texttt{tax}(i, \texttt{price}(i))$$

and, for every screen displaying a total, assert that the displayed total equals the computed total for the item in the cart. Formally this is a **metamorphic relation** — with two items it generalises to $\texttt{total}(\{i,j\}) = \texttt{total}(\{i\}) + \texttt{total}(\{j\})$ under additive pricing. It is a pure function over the pricing module, requires no UI, and fails the build.

This assertion would not have caught the genuine defect of Section III-D, because the arithmetic there is correct. But a *stricter* property any implementation must satisfy would be: for every displayed percentage r beside a displayed amount a and base b, assert $a = \mathrm{round}(r \times b, 2)$ **for the base the label implies**. A property-based test enumerating the catalogue over every rendering site catches that defect at build time, with no human reading a string. We make the general point precisely because our headline case was refuted: **the checks that matter here are cheap, mechanical, and belong in the pipeline.**

D. The audit procedure

For teams without a build pipeline capable of running such assertions:

1) **Enumerate every displayed monetary value** — unit prices, subtotals, tax lines, fees, discounts, deposits.
2) **Reconstruct each value from the others** on any screen showing more than one; where a total cannot be derived, check whether an unstated component is labelled.
3) **Check internal rate statements.** A displayed percentage must match the displayed amount and the base the label implies (Section III-D).
4) **Cross-check screens.** The same service should not render differently across home, catalogue, detail, and invoice screens.
5) **Test at cart boundaries.** Add, remove, or swap items and check whether the total responds.
6) **Exercise non-default paths** — discounts, coupons, and multi-item carts.

Step 0, which this paper adds. Before any of the above, **establish that the figures you are auditing came from a build that exists.** Section III is the counterexample: six steps of sound procedure applied to figures whose provenance was never established.

E. Threats to validity

T-1 — Static inspection, not executed reproduction. [Moderate] We read the pricing module rather than launching the prototype. The arithmetic result is solid, but UI-layer transformation was not exercised.

T-2 — Build identity unestablished. [Severe] No session record carries a build hash, date, or moderator. The two confirmed $100.98 observations cannot be tied to a specific version.

T-3 — Rosters do not agree. [Moderate] The narrative and SEQ corpora have partially disjoint rosters, and the corroborating $100.98 observations come from different corpora than the $198.00 ones. We treat them as independent because they are separately recorded; a single reconciled roster could weaken that.

T-4 — The mechanism of the failed transcription is unknown. [Moderate] We can say $198.00 was not produced by any build and that the two sessions reported a mutually consistent set; the cause (misreading, contamination, or another screen) is unknown. The verification requirement holds regardless.

Other threats. T-5: the ambiguous base may occur on other screens. T-6: $n = 10$, one prototype, one category. T-8: references partly unverified. T-9: the method is not yet externally validated.

V. Conclusion

We set out to report a Critical defect: a marketplace whose stated purpose is transparent pricing displayed a total of $198.00 whose own line items sum to $111.50. **That finding was wrong, and we withdraw it.** The prototype computes totals by pure derivation, and for the $89 service it renders $100.98. The $198.00 appears in no build; both failures trace to the same two sessions.

The procedure then found a real defect: the interface labels its tax line `Tax (8%)` while computing tax on the subtotal including the service fee.

The methodological finding is durable: **numeric claims extracted from usability sessions are measurements, and measurements require verification before they are reported.** Code inspection analyses controls while the defect lives in the relation between them; usability testing analyses journeys while the participant reaches confirmation.

Price-integrity auditing is a build invariant plus a short manual procedure, and because the defect class is about relations between displayed values, it belongs to the metamorphic and property-based testing family. We also propose treating transparency as a property of information consistency. And we propose a Step 0: establish that the build exists.

Acknowledgment

Meutia Braniwati chaired the study; Aditya Paramananda and Tan, Michael Wilson Setiawan designed the prototype; Stephanie Evangelina Pramudito handled administration and visual design.

Author Contributions

CRediT: Bagaskoro Saputro (Methodology, Investigation, Formal analysis, Writing); Yashella Tirana (Formal analysis, Writing); Freysia Chandra Saliman (Investigation, Data curation, Resources, Writing); Laurentius Ryan Dofani Pradita, Gregory Adrianus Sugiono, Gregorius Andrew Rahardja, Steven Karunia Liem, and Joshua Joyson Kustiadi (Investigation, Data curation).

Data Availability

Data deposited on OSF: reproduction procedure, worked arithmetic, source excerpt, and classification. Recordings are not shared and are deleted on 1 January 2027.

References

[1] A. M. Tianti, B. N. Firmania, Prini, R. C. P. Fanani, S. E. Putri, and K. A. Adisusilo, "Peningkatan promosi usaha catering UMKM melalui pembuatan website berbasis WordPress," *PENITI BANGSA*, vol. 2, no. 2, pp. 1–8, 2024. doi: 10.30742/PENITI-BANGSA.v2i22024.177

[2] P. Z. Dinata, M. A. Urwah, M. R. Rahmawan, and E. Junaeti, "Perancangan UI/UX web e-commerce 'Hallo Coffee' menggunakan metode *user-centered design*," *Jambura J. Inform.*, vol. 5, no. 1, pp. 45–58, 2023. doi: 10.37905/jji.v5i1.17511

[3] B. Sunarso, Tusriyanto, and F. Mustafa, "Analysing the role of visual content in increasing attraction and conversion in MSME digital marketing," *J. Contemp. Adm. Manag. ADMAN*, vol. 1, no. 3, pp. 193–200, 2023. doi: 10.61100/adman.v1i3.79

[4] J. Sauro and J. R. Lewis, *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed. Sebastopol, CA: Morgan Kaufmann, 2016.

[5] J. Nielsen, "Enhancing the explanatory power of usability heuristics," in *Proc. ACM CHI'94*, 1994, pp. 152–158. doi: 10.1145/191666.191729

[6] A. R. Perdanakusuma, B. T. Hanggara, and A. R. Hasnanursanti, "Analisis usability website resmi pemerintah kota *Surakarta* menggunakan metode heuristic evaluation," *J. TECNOSCIENZA*, vol. 6, no. 2, pp. 430–443, 2022.

[7] A. Febriyanthi, "Interface design heuristic evaluation website yayasan kesehatan Telkom," *Turk. J. Comput. Math. Educ. TURCOMAT*, vol. 12, no. 4, pp. 852–859, 2021.

[8] W. Y. Chen and A. Meier, "Program self-verification by dynamic program analysis," Technical Report LC-1789, Lawrence Livermore National Laboratory, 2003.

[9] J. Claessen and D. McAllister, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proc. ICFP'00*, 2000, pp. 346–359. doi: 10.1145/351240.351266

[10] Software Engineering Institute, *The capability maturity model for software development (integrated)/CMMI-SMD*. Pittsburgh, PA: SEI, 2002.

[11] M. Braniwati, "Aggregate task success hides the signal: Per-task dispersion and ceiling effects in formative usability evaluation," companion paper, same study, unpublished.

[12] D. Khaiurov, "The psychology of fee disclosure: How drip pricing violates fairness norms and undermines consumer trust in e-commerce checkout interfaces," SSRN Electronic Journal, 2026. **[Verify citation.]**

[13] B. Lu, D. Markovits, A. Miller, and R. Van Loo, "Measuring and mitigating drip pricing overcharge: Evidence from an online marketplace experiment with a digital shopping assistant," 2025. **[Verify citation.]**

[14] K. K. Kim, Y. Ryoo, S. Zdravković, and S. Yoon, "Price transparency in international retailing on digital platforms," *Int. Mark. Rev.*, 2024. doi:10.1108/imr-12-2023-0352

[15] D. Totzek and G. Jurgensen, "Many a little makes a mickle: Why do consumers negatively react to sequential price disclosure?" *Psychol. Mark.*, 2021. doi:10.1002/mar.21426

[16] T. Willberg-Laine, "Testing graphical user interfaces with property-based testing," 2022. **[Verify citation.]**

[17] A. Delgado, A. Estepa, J. Troyano, and R. Estepa, "Assessing behavioral validation in UI component test suites using inferred metamorphic relations," arXiv, 2026. **[Verify citation.]**